

Minor Assignment 07 — Solutions

Game Programming with C++ (CSE 3545) • Bullet class • Collision detection • Inheritance & Polymorphism

This assignment finishes the Zombie Arena core loop: **shoot bullets** with a real trajectory, **detect collisions** (bullet↔zombie, zombie↔player), and switch game state when the round ends. The last few questions are pure C++ inheritance theory — public / private / protected derivation and multiple inheritance.

Three big ideas:

- 1. Bullet trajectory** — a bullet has a fixed starting point, a fixed direction (computed from the crosshair), and a fixed maximum range. Once fired, it travels on its own until it hits something or runs out of range.
- 2. Collision = rectangle intersection** — every drawable has a bounding box (`getGlobalBounds()`). Two boxes overlap → collision. Simple, fast, good enough for 2D.
- 3. Inheritance access** — the keyword (public/private/protected) on the derivation line decides how the base class's members appear inside and outside the derived class.

Q1. SFML class to represent a bullet

A bullet is small, axis-aligned, single-colour. The simplest SFML drawable that fits is `sf::RectangleShape` — no texture file needed, just set its size and fill colour.

```
class Bullet{
private:
    RectangleShape m_BulletShape;    // <-- visual representation of the bullet
    // ... other members (position, speeds, in-flight flag) ...
};
```

Why RectangleShape and not Sprite? Sprites need a texture file (bullet.png). For a 20×20 solid-coloured projectile, RectangleShape is lighter — no PNG to load, no Texture to manage.

Q2. Public member function prototypes of Bullet

The Bullet class needs to be: constructed, fired toward a target, stopped when it hits something, drawn, and queried by the game (is it still flying? where is it? what's its bounding box for collisions?).

```
class Bullet{
private:
    Vector2f      m_Position;
    RectangleShape m_BulletShape;

    bool  m_InFlight = false;
    float m_BulletSpeed    = 1000;    // px / second
    float m_BulletDistanceX;          // dx per frame
    float m_BulletDistanceY;          // dy per frame
    float m_MaxX, m_MinX;
    float m_MaxY, m_MinY;

public:
    Bullet();                          // constructor

    void stop();                       // call when bullet hits or
                                      // leaves range
    bool isInFlight() const;           // is it still alive?

    void shoot(float startX, float startY,
               float targetX, float targetY);    // fire toward a target

    FloatRect getPosition() const;      // bounding box for
                                      // collision checks
    RectangleShape getShape() const;    // for window.draw()
    void update(float elapsedTime);      // advance the bullet
};
```

```

}; // each frame

```

Q3. Constructor — 20 × 20 red bullet

```

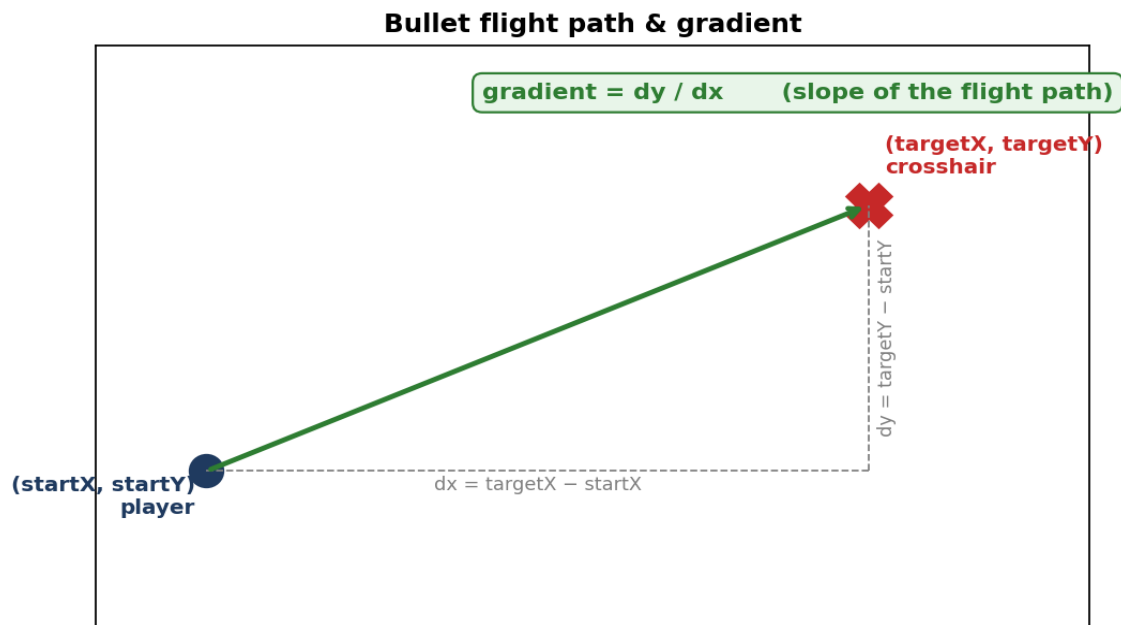
Bullet::Bullet(){
    m_BulletShape.setSize(Vector2f(20.0f, 20.0f)); // size = 20 x 20
    m_BulletShape.setFillColor(Color::Red);        // colour = red
}

```

Tip: for a much smaller, more realistic bullet you can use `setSize(Vector2f(2.f, 2.f))`. The assignment specifies 20×20, so we use that.

Q4. Set bullet position + compute flight gradient

When a bullet is fired, the player's current location becomes its starting point. The crosshair location is the target. The **gradient** is the slope of the straight line from start to target: dy / dx .



```

// Set the bullet's starting position
m_Position.x = startX;
m_Position.y = startY;

// Calculate the gradient (slope) of the flight path
float gradient = (startY - targetY) / (startX - targetX);

// Avoid extreme/undefined slopes (vertical shots).
// In the textbook code, the gradient is clamped:
if (gradient < 0) {
    gradient *= -1; // make it positive for the speed split below
}

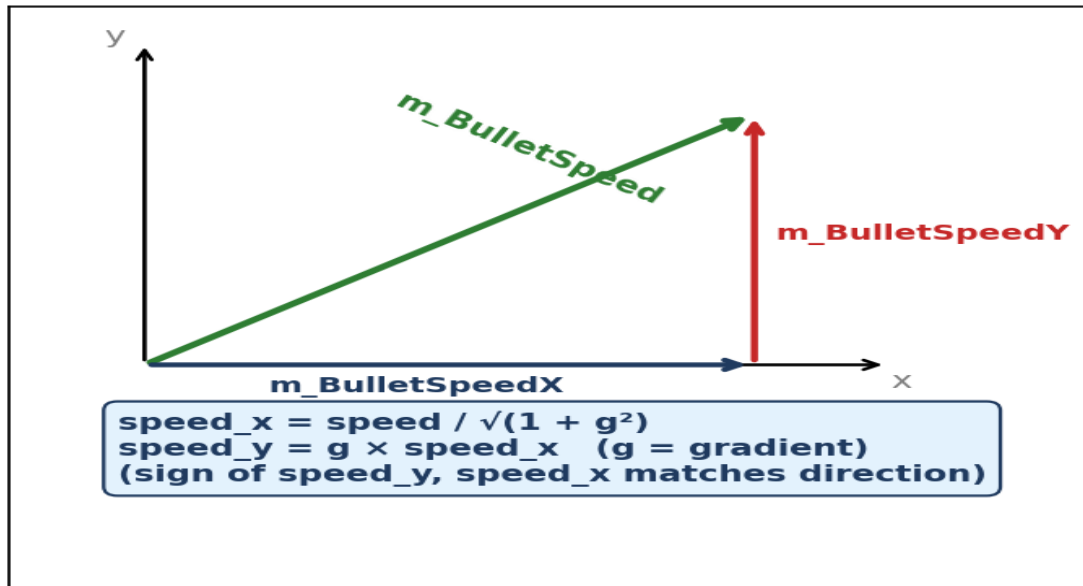
```

Why take absolute value? The *magnitude* of the gradient drives speed splitting; the *direction* (left/right, up/down) is decided separately by comparing startX/Y with targetX/Y.

Q5. Decompose bullet speed using gradient

The bullet has one total speed `m_BulletSpeed` (say 1000 px/s). We split it into x and y components so that the *ratio* matches the gradient, and the *combined* speed still equals `m_BulletSpeed`.

Splitting bullet speed into x & y components



From the geometry (Pythagoras):

```
// gradient = dy / dx, so dy = gradient * dx
// Combined speed: speed^2 = vx^2 + vy^2 = vx^2 (1 + gradient^2)
// Therefore:      vx = speed / sqrt(1 + gradient^2)
//               vy = gradient * vx
```

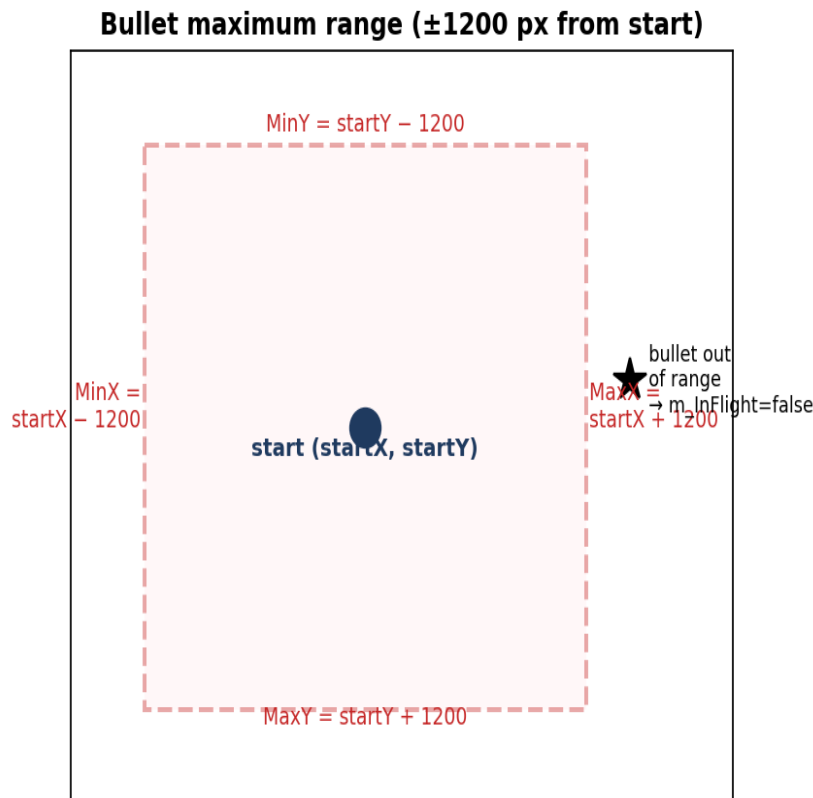
```
m_BulletSpeedX = m_BulletSpeed / sqrt(1 + (gradient * gradient));
m_BulletSpeedY = gradient * m_BulletSpeedX;
```

```
// Then fix the SIGN so the bullet moves in the correct direction:
if (targetX < startX) m_BulletSpeedX *= -1;
if (targetY < startY) m_BulletSpeedY *= -1;
```

Renaming hint from the question: the same two variables are also called $m_BulletDistanceX$ and $m_BulletDistanceY$ — they represent how far the bullet moves on each axis per frame.

Q6. Set maximum range (± 1200 px in any direction)

Without a range cap, bullets fly forever, slowing the game. We compute a **bounding box** centred on the starting position, 1200 px on each side. As soon as the bullet's position leaves this box, it's stopped.



```
// Set a max range of 1200 pixels in any direction
m_MaxX = startX + 1200;
m_MinX = startX - 1200;
m_MaxY = startY + 1200;
m_MinY = startY - 1200;

// Place the bullet shape at the starting position and mark it active
m_BulletShape.setPosition(startX, startY);
m_InFlight = true;
```

Q7. Test whether the bullet has gone past its max range

Inside `update()` we advance the bullet, then check all four boundaries. Any one of them being crossed → deactivate the bullet.

```
void Bullet::update(float elapsedTime){
    // 1. Move the bullet
    m_Position.x += m_BulletSpeedX * elapsedTime;
    m_Position.y += m_BulletSpeedY * elapsedTime;
    m_BulletShape.setPosition(m_Position);

    // 2. Has it gone beyond its max range? If so, stop it.
    if (m_Position.x < m_MinX || m_Position.x > m_MaxX ||
        m_Position.y < m_MinY || m_Position.y > m_MaxY) {
        m_InFlight = false;
    }
}
```

Q8. Fire a bullet on left mouse button click

We don't use `Mouse::isButtonPressed` (continuous polling) here — that would fire dozens of bullets per click. Instead we use the **event** `MouseButtonPressed`, which fires exactly once per click. This belongs inside the **event-polling loop** in `main`, alongside the key-press handlers.

```
// ---- inside the main event loop ----
while (window.pollEvent(event)){

    if (event.type == Event::Closed)
        window.close();

    if (event.type == Event::KeyPressed) { /* state changes */ }

    // ---- LEFT MOUSE BUTTON → FIRE A BULLET ----
    if (state == State::PLAYING) {
        if (event.type == Event::MouseButtonPressed) {
            if (event.mouseButton.button == Mouse::Left) {

                if (bulletsInClip > 0) {
                    // shoot bullet at index currentBullet
                    bullets[currentBullet].shoot(
                        player.getCenter().x, player.getCenter().y,
                        mouseWorldPosition.x, mouseWorldPosition.y);

                    currentBullet++;
                    if (currentBullet > 99) currentBullet = 0;

                    lastPressed = gameTimeTotal;
                    shoot.play();          // play shoot.wav
                    bulletsInClip--;
                }
                else if (clipSize > 0 && spareAmmo > 0) {
                    // auto-reload when clip is empty
                    reload.play();
                }
                else {
                    reloadFailed.play();
                }
            }
        }
    }
}
```

Where this code lives: inside the `while (window.pollEvent(event))` block in `main()`, after the keyboard handling. It only fires when the current state is `PLAYING`.

Q9. Loop through bullets array and draw active ones

We keep a fixed-size array (typically 100 bullets) that gets recycled. Each frame, draw *only* the ones currently in flight.

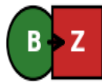
```
// Assume:  Bullet bullets[100];

for (int i = 0; i < 100; i++) {
    if (bullets[i].isInFlight()) {
        window.draw(bullets[i].getShape());
    }
}
```

Why a fixed array? Allocating/freeing memory every shot would stutter the game. The array of 100 acts as an object pool — `currentBullet` wraps around 0..99, reusing slots.

Q10. Zombie touches the player → reduce health, maybe GAME_OVER

Three collision cases handled in Zombie Arena



Bullet vs Zombie
→ zombie hit



Zombie vs Player
→ player loses health



Bullet out of range
→ `m_inFlight = false`

```
// For every live zombie, test its bounding box against the player's
for (int i = 0; i < numZombies; i++) {

    if (player.getPosition().intersects(zombies[i].getPosition())
        && zombies[i].isAlive()) {

        // Player has been touched → reduce health
        if (player.hit(gameTimeTotal)) {
            // sound effect (optional): hit.play();
        }

        // If health drops to 0 or below → GAME OVER
        if (player.getHealth() <= 0) {
            state = State::GAME_OVER;
            // ...save high score, switch screens, etc.
        }
    }
}
```

How intersects works: `FloatRect::intersects(other)` returns *true* if any pixel of the two rectangles overlap. It's an AABB (axis-aligned bounding box) test — fast and good enough for 2D.

Q11. Bullet hits a zombie → kill it; all dead → LEVELING_UP

Two nested loops: every in-flight bullet against every alive zombie. On hit, stop the bullet and kill the zombie. After the loop, count remaining zombies — zero left means the wave is cleared.

```
// 1. Detect bullet ↔ zombie collisions
for (int i = 0; i < 100; i++) {                // every bullet slot
    for (int j = 0; j < numZombies; j++) {    // every zombie
```

```

    if (bullets[i].isInFlight() && zombies[j].isAlive()) {
        if (bullets[i].getPosition().intersects(
            zombies[j].getPosition())) {

            bullets[i].stop();                // bullet done
            if (zombies[j].hit()) {            // zombie loses HP
                score += 10;
                if (score >= hiScore) hiScore = score;
                numZombiesAlive--;             // one fewer alive

                // optional: splat.play();
            }
        }
    }
}

// 2. All zombies dead? → next wave / level up screen
if (numZombiesAlive == 0) {
    state = State::LEVELING_UP;
}

```

Q12. public derivation — count B's usable public members

```
class A {
public:
    void f1();
    void f2();
    void f3();
    void f4();
};

class B : public A {           // <-- PUBLIC derivation
public:
    void g1();
};
```

Under **public** derivation, every public member of A stays public in B. So an outside user of B can call:

- A's 4 public members (f1, f2, f3, f4) — inherited as public
- B's own 1 public member (g1)

Total public members usable through a B object = 4 + 1 = 5.

Q13. private derivation — count B's usable private members

```
class A {
public:
    void f1();
    void f2();
    void f3();
    void f4();
};

class B : private A {         // <-- PRIVATE derivation
private:
    void g1();                // B's own 1 private member
};
```

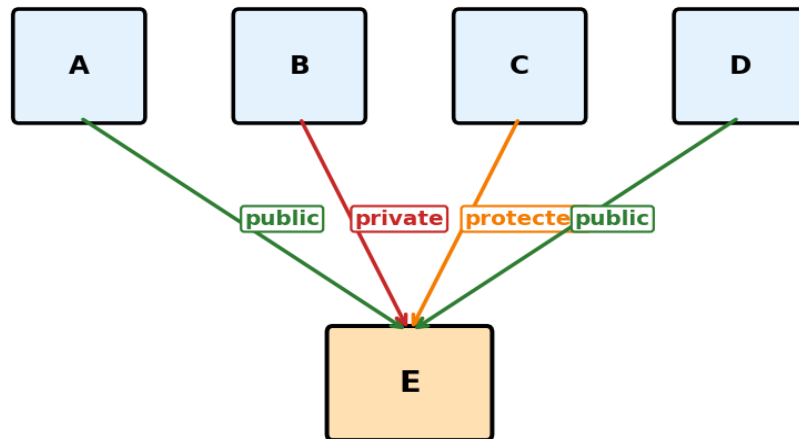
Under **private** derivation, A's public members become *private* inside B. They are still inherited (B's member functions can call them), but the outside world cannot. Adding B's own private member:

- A's 4 public members → now private inside B
- B's own 1 private member

Total private members usable inside B = 4 + 1 = 5.

Q14. Multiple inheritance — A (public), B (private), C (protected), D (public)

Q14: class E inherits A (public), B (private), C (protected), D (public)



```
class A { /* ... */ };
class B { /* ... */ };
class C { /* ... */ };
class D { /* ... */ };

// E inherits from all four with different access specifiers
class E : public A, private B, protected C, public D {
public:
    // ... E's own members ...
};
```

How access specifiers transform under each inheritance type

Base member	public derivation	protected derivation	private derivation
public	→ public	→ protected	→ private
protected	→ protected	→ protected	→ private
private	× not inherited	× not inherited	× not inherited

What each derivation means for E's interface:

- **public A** — A's public stays public, protected stays protected
- **private B** — all of B's accessible members become private in E (hidden from outside)
- **protected C** — C's public & protected become protected in E (visible to further subclasses, not outside)
- **public D** — same rule as A: public stays public

Q15. Output of the multi-inheritance constructor trace

First, the code in the question has two bugs that must be fixed before it compiles: (a) the derived class is missing a name — should be `class Derived`, and (b) the constructor inside `Derived` has return type `public:D()` which should be `Derived()`. Once fixed, the code is a textbook multi-inheritance construction-order question.

Corrected code:

```
class B1 {
public:
    B1(){ cout << "B1" << endl; }
};
class B2 {
public:
    B2(){ cout << "B 2" << endl; }
};
class Derived : public B1, public B2 {
public:
    Derived(){ cout << "D" << endl; }
};
int main(){
    Derived d;
    return 0;
}
```

Rule for construction order in multiple inheritance:

Base-class constructors run in the order they appear in the *derivation list* (left to right), not in the order their objects are declared. The derived class's own constructor runs last. So for `class Derived : public B1, public B2`:

1. `B1()` runs → prints **B1**
2. `B2()` runs → prints **B 2**
3. `Derived()` runs → prints **D**

Output:

```
B1
B 2
D
```

Destructor order is the reverse: if these had destructors, on object destruction you'd see `D → B2 → B1`.

Quick reference — cheat sheet for the exam

Bullet pipeline (the whole flow in 6 steps)

1. Constructor sets size + colour.
2. `shoot(startX, startY, targetX, targetY)` → sets position, computes gradient, splits speed into `m_BulletSpeedX` / `m_BulletSpeedY`, sets `m_InFlight = true`.
3. Each frame, `update(dt)` advances the position and checks the range box.
4. Out of range → `m_InFlight = false`.
5. Main loop draws every in-flight bullet.
6. Collision test stops bullets that hit zombies.

Trajectory math

```
gradient      = (startY - targetY) / (startX - targetX); // sign-free in code
m_BulletSpeedX = m_BulletSpeed / sqrt(1 + gradient * gradient);
m_BulletSpeedY = gradient * m_BulletSpeedX;
// then fix signs based on target direction
```

Collision idiom

```
if (A.getPosition().intersects(B.getPosition())) {
    // they overlap – handle hit
}
```

Inheritance access — memorise this table

How access specifiers transform under each inheritance type

Base member	public derivation	protected derivation	private derivation
public	→ public	→ protected	→ private
protected	→ protected	→ protected	→ private
private	× not inherited	× not inherited	× not inherited

Multi-inheritance construction order

Base constructors run **left to right** in the derivation list, then the derived constructor runs. Destructors run in reverse order.

Viva tip: if asked the difference between `Mouse::isButtonPressed` and the `MouseButtonPressed` event — the first is *state polling* (true every frame the button is held), the second is an *event* (fires once per click). For firing bullets, you want the event so one click = one bullet.